

National University of Computer and Emerging Sciences



Lab Manual 09 Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 9	4
Objectives:.....	4
Activities:	4
Activity 1 (Machine Learning Concepts):	4
1. What is Machine Learning	4
Types of Machine Learning	4
• Supervised Learning	5
Types of Supervised Learning	5
i. Classification	5
ii. Regression	5
2. What is Scikit-learn?	5
Activity 2 (Examples):	8
• Classification	8
○ Import Libraries	8
○ Load the Dataset	9
○ Split Data (Train/Test)	9
○ Feature Scaling	9
○ Train the Classifier	9
○ Make Predictions	9
○ Evaluate the Model	9
○ Trying Another Classifier	10
• Regression	10
○ Import Libraries	10
○ Load the Dataset	10
○ Split Data (Train/Test)	10
○ Feature Scaling	10
○ Train the Regression Model	10
○ Make Predictions	11
○ Evaluate the Model	11
○ Trying Another Regression Model	11
• Evaluation metrics for Regression	11
○ Mean Absolute Error (MAE)	11
○ Mean Squared Error (MSE)	12
○ Root Mean Squared Error (RMSE)	12

○ R-squared (R^2) Score	12
● Evaluation metrics for Classification	13
○ Confusion Matrix	13
○ Accuracy	14
○ Precision	14
○ Recall.....	15
○ F1 Score	15
○ Area Under Curve (AUC) and ROC Curve.....	15
○ Classification Report	16
Activity 3 (Problem Statements / Tasks):	17
1. Iris Classification with Model Comparison.....	17
2. Effect of Feature Scaling	17
3. Regression Model Comparison	17
4. Understanding Evaluation Metrics (Classification)	18
5. ROC Curve & Model Performance	18
6. Train-Test Split Importance.....	18

Lab Manual 9

Objectives:

- ✓ Machine Learning
- ✓ Scikit-Learn
- ✓ Classification
- ✓ Regression

Activities:

Activity 1 (Machine Learning Concepts):

1. *What is Machine Learning*

Machine Learning (ML) is a technique that allows computers to learn patterns from data and make decisions or predictions without being explicitly programmed. Instead of writing rules manually, ML algorithms learn rules automatically from data.

Key Idea:

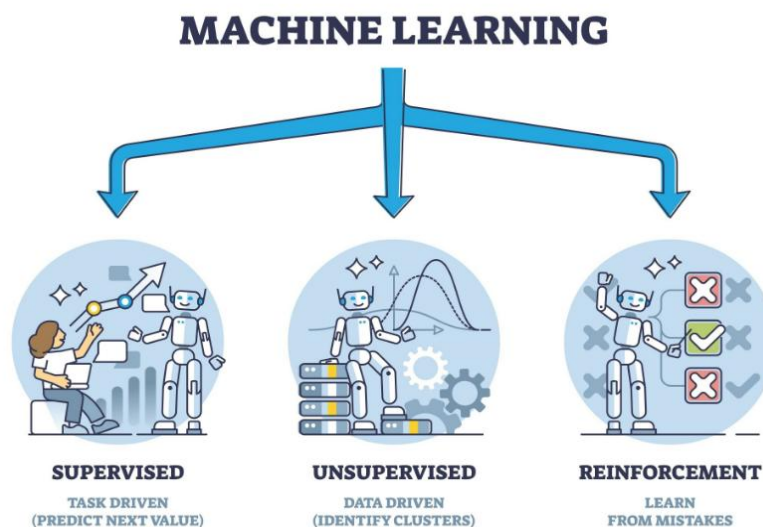
- Traditional Programming → Rules + Data = Output
- Machine Learning → Data + Output = Model learns rules

Examples:

- Spam detection
- Disease prediction
- Recommendation systems
- Price prediction

Types of Machine Learning

Type	Description	Example
Supervised Learning	Data has labels	Predict marks
Unsupervised Learning	No labels	Customer grouping
Reinforcement Learning	Learning via rewards	Game AI



- *Supervised Learning*

Supervised learning uses:

- Input (X) → Features
- Output (y) → Target

Model learns a function:

$$f(X) = y$$

Key Terms:

- Feature → Input variable
- Target → Output variable
- Dataset: Collection of samples
- Model → Algorithm that learns function
- Training → Learning phase from known data
- Testing → Evaluation phase on unseen data

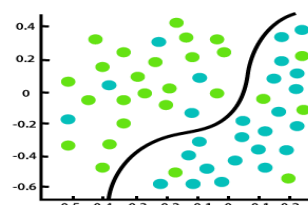
Types of Supervised Learning

i. *Classification*

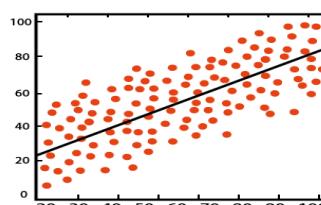
- Output is categorical
- Example:
 - Spam / Not Spam
 - Pass / Fail
- Goal: Assign input to a class

ii. *Regression*

- Output is continuous
- Example:
 - House price
 - Temperature
- Goal: Predict numeric value



Classification



Regression

2. *What is Scikit-learn?*

Scikit-learn is a Python library used for building machine learning models easily.

Why use it?

- No need to write complex math
- Easy model training
- Built-in evaluation tools

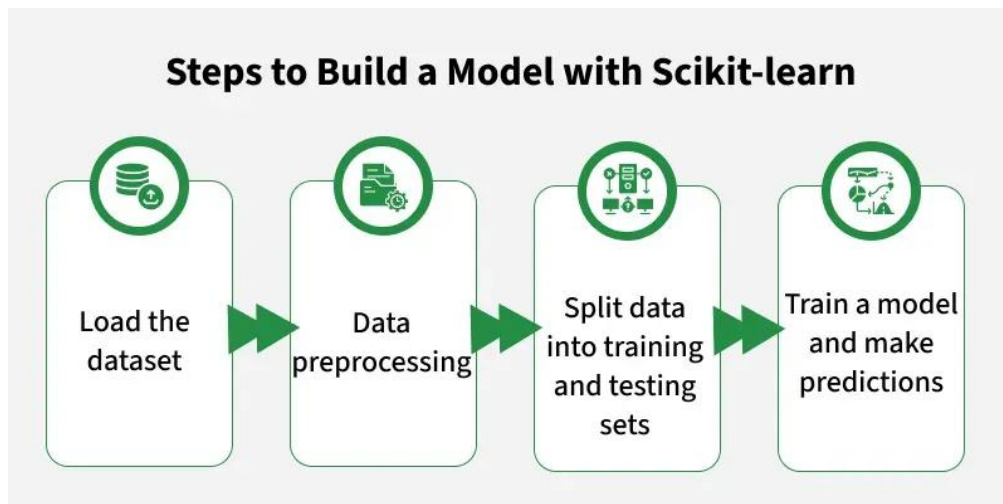
Built on:

- NumPy
- Pandas
- Matplotlib

Basic Workflow

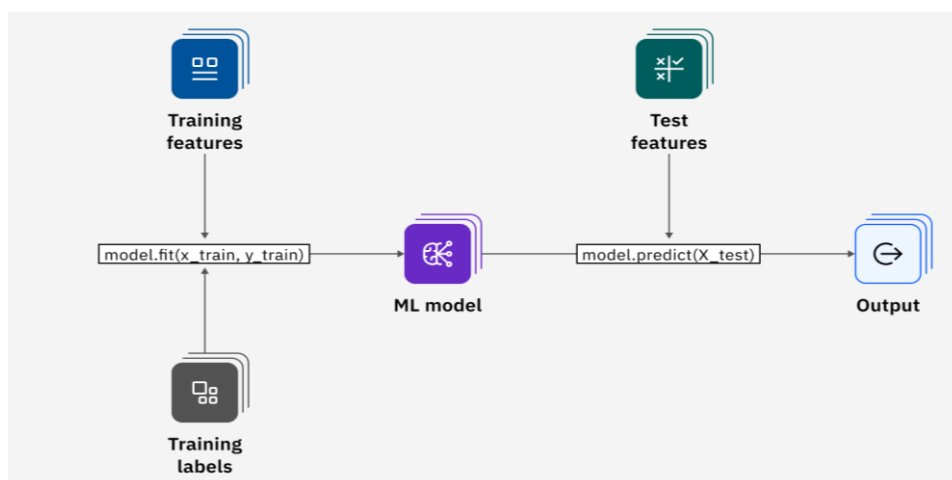
- Load dataset
- Split data (Train/Test)
- Scale features

- Train model
- Predict
- Evaluate



Important Functions

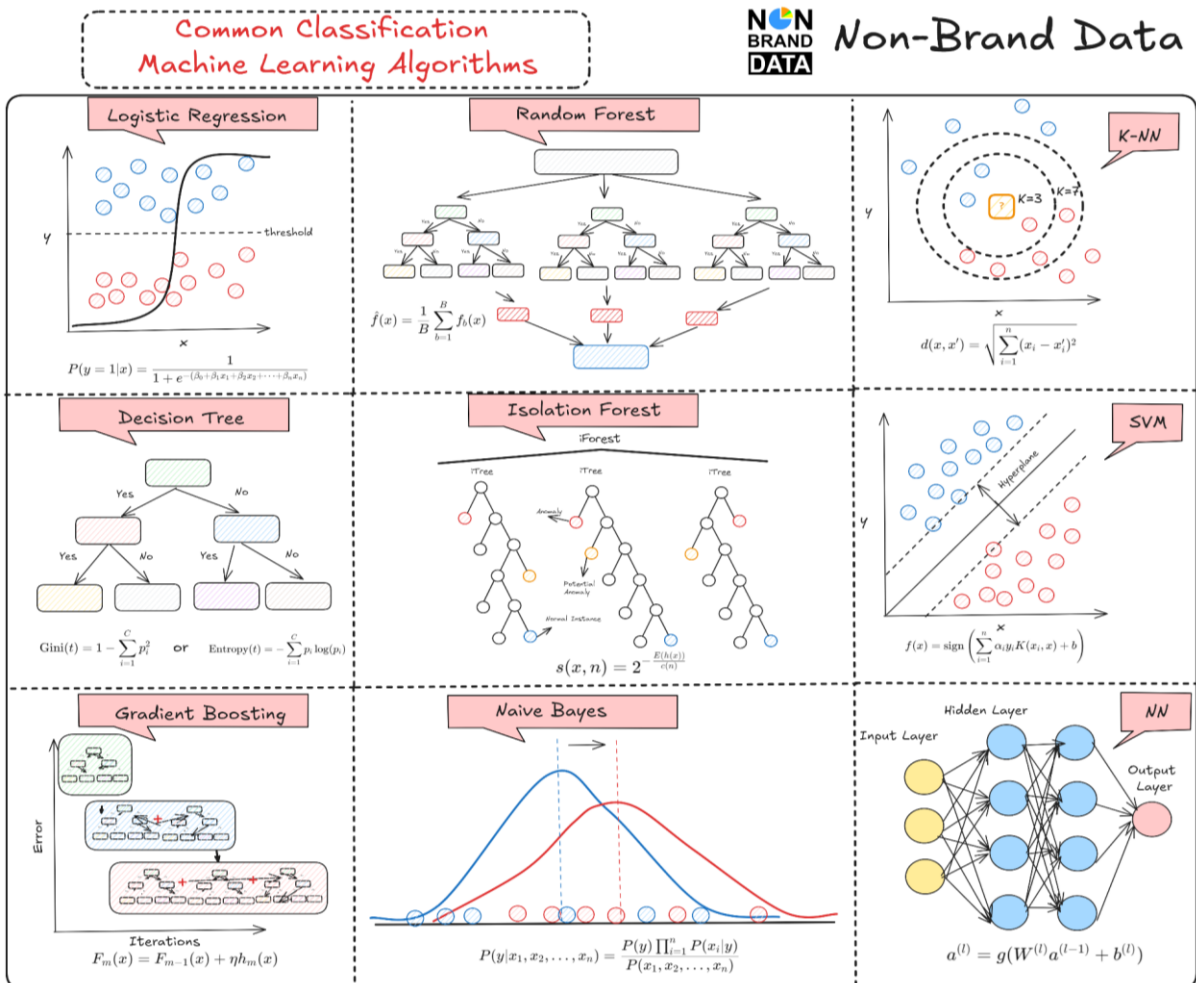
Function	Purpose
fit()	Train model
predict()	Predict output
score()	Evaluate model



Scikit-learn supports many machine learning tasks such as:

- Supervised Learning
 - Linear Regression
 - Logistic Regression
 - K-Nearest Neighbors
 - Decision Trees
 - Random Forest
 - Support Vector Machines
- Unsupervised Learning
 - K-Means Clustering
 - Hierarchical Clustering

- PCA (Dimensionality Reduction)
- Preprocessing Tools
 - Scaling features
 - Encoding categorical variables
 - Handling missing values
- Model Evaluation
 - Accuracy
 - Precision, Recall
 - Confusion Matrix
 - Cross-validation



Split Data (Train/Test)

In machine learning, our goal is not just to build a model that performs well on known data, but one that performs well on new, unseen data.

If we train and test the model using the same dataset, the model may memorize the data and the performance may look very high but it may fail in real-world scenarios.

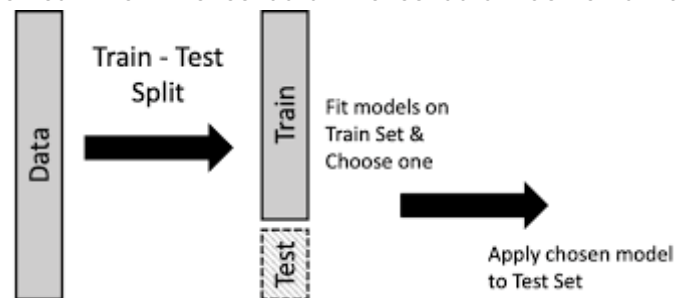
To avoid this, we divide the dataset into two parts:

- Training Set: The training set is used to:
 - Teach the model
 - Learn patterns
 - Estimate parameters (weights)

During training, the model:

- Looks at input features (X)
- Compares predictions with actual outputs (y)
- Adjusts its parameters to reduce error
- Testing Set: The testing set is used to:
 - Evaluate model performance
 - Measure generalization ability
 - Simulate real-world prediction

The model must never learn from the test data. The test data must remain unseen during training.



Suppose we have 100 student records. If we split 80–20:

- 80 records: Training set
- 20 records: Testing set

The model learns from the 80 students and is evaluated on the 20 new students.

Python Code:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)
```

Parameter	Meaning
X	Features
y	Target
test_size=0.2	20% data used for testing
random_state=42	Ensures reproducibility

When splitting data:

- The split is random.
- Different runs may give different results.

By setting random_state=42, we ensure:

- Same split every time
- Reproducible results
- Fair comparison between models

Activity 2 (Examples):

- *Classification*

- Import Libraries

```
import numpy as np
import pandas as pd
```

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
                                classification_report

```

○ Load the Dataset

```
iris = load_iris()
```

```

X = iris.data      # Features
y = iris.target    # Labels

```

```

print(X.shape)    # (150 samples, 4 features)
print(y.shape)    # (150,)

```

○ Split Data (Train/Test)

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

```

○ Feature Scaling

```
scaler = StandardScaler()
```

```

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

○ Train the Classifier

```

model = LogisticRegression()
model.fit(X_train, y_train)

```

We are using logistic regression as a classifier here. You can explore other classifiers here: https://scikit-learn.org/stable/supervised_learning.html

What happens internally:

- The model learns decision boundaries
- It finds weights that best separate classes

○ Make Predictions

```
y_pred = model.predict(X_test)
```

○ Evaluate the Model

```

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

```

```

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:", cm, sep="\n")

```

```

print("Classification Report:",
      classification_report(y_test, y_pred), sep="\n")

```

- Trying Another Classifier

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

- Regression

- Import Libraries

```
import numpy as np
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

- Load the Dataset

```
data = fetch_california_housing()

X = data.data      # Features
y = data.target    # Target

print(X.shape)     # (20640 samples, 8 features)
print(y.shape)     # (20640,)
```

- Split Data (Train/Test)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

- Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

- Train the Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

We are using linear regression as a regressor here. You can explore other regressors here: https://scikit-learn.org/stable/supervised_learning.html

What happens internally:

- Model finds weights for each feature
- Minimizes the difference between predicted and actual values

- **Make Predictions**

```
y_pred = model.predict(X_test)
```

- **Evaluate the Model**

```
mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
```

```
rmse = np.sqrt(mse)
print("RMSE:", rmse)
```

```
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

- **Trying Another Regression Model**

```
from sklearn.tree import DecisionTreeRegressor

tree_model = DecisionTreeRegressor(random_state=42)
tree_model.fit(X_train, y_train)

y_pred_tree = tree_model.predict(X_test)

print("Decision Tree R2 Score:", r2_score(y_test, y_pred_tree))
```

- **Evaluation metrics for Regression**

Regression models predict continuous outcomes (e.g., house prices, temperatures). To measure their performance, we need evaluation metrics that quantify how close predictions are to actual values. There is no single “best” metric; each has strengths and weaknesses.

- **Mean Absolute Error (MAE)**

In the fields of statistics and machine learning, the Mean Absolute Error (MAE) is a frequently employed metric. It's a measurement of the typical absolute discrepancies between a dataset's actual values and projected values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |x_i - y_i|$$

where:

- x_i represents the actual or observed values for the i -th data point.
- y_i represents the predicted value for the i -th data point.

```
from sklearn.metrics import mean_absolute_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

mae = mean_absolute_error(true_values, predicted_values)
print("Mean Absolute Error:", mae)
```

○ Mean Squared Error (MSE)

A popular metric in statistics and machine learning is the Mean Squared Error (MSE). It measures the square root of the average discrepancies between a dataset's actual values and projected values. MSE is frequently utilized in regression issues and is used to assess how well predictive models work.

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2$$

where:

- x_i represents the actual or observed values for the i-th data point.
- y_i represents the predicted value for the i-th data point.

```
from sklearn.metrics import mean_squared_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

mse = mean_squared_error(true_values, predicted_values)
print("Mean Squared Error:", mse)
```

○ Root Mean Squared Error (RMSE)

RMSE stands for Root Mean Squared Error. It is a usually used metric in regression analysis and machine learning to measure the accuracy or goodness of fit of a predictive model, especially when the predictions are continuous numerical values.

$$RMSE = \sqrt{MSE}$$

```
from sklearn.metrics import mean_squared_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

rmse = np.sqrt(mean_squared_error(true_values,
predicted_values))
print("Root mean Squared Error:", rmse)
```

○ R-squared (R^2) Score

A statistical metric frequently used to assess the goodness of fit of a regression model is the R-squared (R^2) score, also referred to as the coefficient of determination. It quantifies the percentage of the dependent variable's variation that the model's independent variables contribute to. R^2 is a useful statistic for evaluating the overall effectiveness and explanatory power of a regression model.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

```
from sklearn.metrics import r2_score

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]
```

```
r2 = r2_score(true_values, predicted_values)
print("R-squared (R2) Score:", r2)
```

- **Evaluation metrics for Classification**

We will be using the following sample dataset for the given example code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_auc_score,
    classification_report,
    roc_curve
)

# Example true labels and predicted labels
y_true = [0, 1, 0, 1, 0, 1, 1, 0]
y_pred = [0, 1, 0, 0, 0, 1, 1, 1]
y_pred_proba = [0.1, 0.9, 0.2, 0.4, 0.3, 0.8, 0.7, 0.6]
# Probabilities for ROC-AUC
```

- **Confusion Matrix**

Confusion matrix creates a N X N matrix, where N is the number of classes or categories that are to be predicted.

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

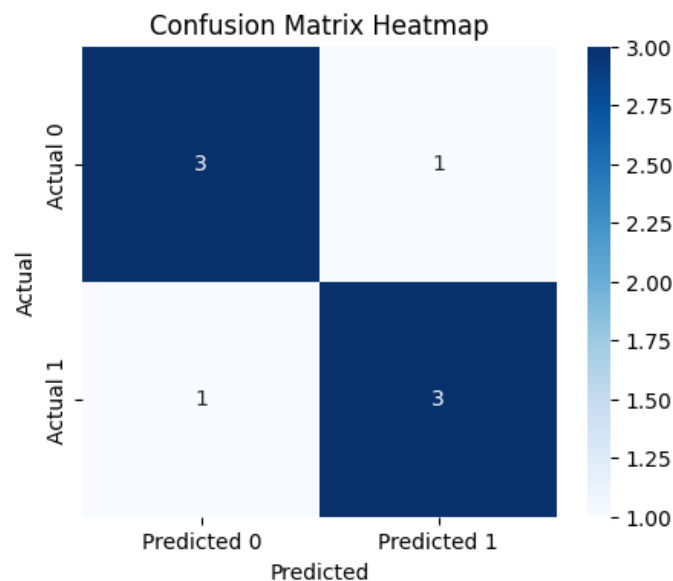
		Actual Value	
		Positive	Negative
Predicated Value	Positive	5600	600
	Negative	500	3300

- True Positives: Correctly predicted positive class.
- True Negatives: Correctly predicted negative class.
- False Positives: Incorrectly predicted positive class (Type I error)
- False Negatives: Incorrectly predicted negative class (Type II error)

```
cm = confusion_matrix(y_true, y_pred)
print("\nConfusion Matrix:\n", cm)
```

To visualize confusion matrix:

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix Heatmap')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```



- Accuracy

Accuracy is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

While accuracy provides a quick snapshot, it can be misleading in cases of imbalanced datasets. For example, in a dataset with 90% class A and 10% class B, a model predicting only class A will still achieve 90% accuracy, but it will fail to identify any class B instances.

Accuracy is good but it gives a False Positive sense of achieving high accuracy. The problem arises due to the possibility of misclassification of minor class samples being very high.

```
accuracy = accuracy_score(y_true, y_pred)
print("Accuracy:", accuracy)
```

- Precision

It measures how many of the positive predictions made by the model are actually correct. It's useful when the cost of false positives is high such as in medical diagnoses where predicting a disease when it's not present can have serious consequences.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision helps ensure that when the model predicts a positive outcome, it's likely to be correct.

```
precision = precision_score(y_true, y_pred)
print("Precision:", precision)
```

- Recall

Recall or Sensitivity measures how many of the actual positive cases were correctly identified by the model. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

In scenarios where catching all positive cases is important (like disease detection), recall is a key metric.

```
recall = recall_score(y_true, y_pred)
print("Recall:", recall)
```

- F1 Score

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number. A high F1 score means the model performs well on both metrics. Its range is [0,1].

Lower recall and higher precision gives us great accuracy but then it misses a large number of instances. More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
f1 = f1_score(y_true, y_pred)
print("F1 Score:", f1)
```

- Area Under Curve (AUC) and ROC Curve

It is useful for binary classification tasks. The AUC value represents the probability that the model will rank a randomly chosen positive example higher than a randomly chosen negative example. AUC ranges from 0 to 1 with higher values showing better model performance.

ROC Curve is a graphical representation of the True Positive Rate (TPR) vs the False Positive Rate (FPR) at different classification thresholds. The curve helps us visualize the trade-offs between sensitivity (TPR) and specificity (1 - FPR) across various thresholds. Area Under Curve (AUC) quantifies the overall ability of the model to distinguish between positive and negative classes.

$$\text{TNR} = \frac{TN}{TN + FP} \quad \text{FPR} = \frac{FP}{FP + TN}$$

AUC = 1: Perfect model (always correctly classifies positives and negatives).

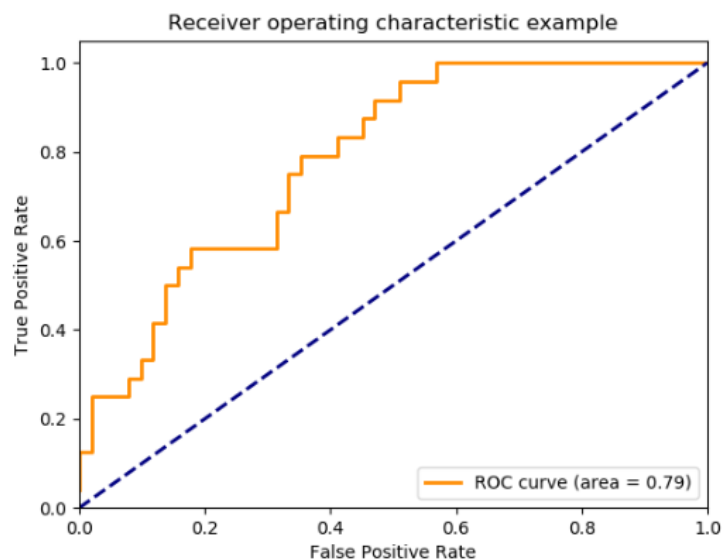
AUC = 0.5: Model performs no better than random guessing.

AUC < 0.5: Model performs worse than random guessing (showing that the model is inverted).

```
roc_auc = roc_auc_score(y_true, y_pred_proba)
print("ROC-AUC Score:", roc_auc)
```

ROC visualization:

```
fpr, tpr, thresholds = roc_curve(y_true, y_pred_proba)
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, color='blue', label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0,1], [0,1], color='red', linestyle='--')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```



- Classification Report

A classification report is a summary table that shows how well your classification model performed, by giving key metrics Precision, Recall, F1-score, and Support for each class in your data.

```
report = classification_report(y_true, y_pred)
print("\nClassification Report:\n", report)
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.75         0.75         0.75         4
     1       0.75         0.75         0.75         4

 accuracy          0.75
 macro avg         0.75         0.75         0.75         8
 weighted avg      0.75         0.75         0.75         8
```

Reference: <https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>

Activity 3 (Problem Statements / Tasks):

1. *Iris Classification with Model Comparison*

Use the Iris dataset to build classification models.

Requirements

- Load dataset using `load_iris()`
- Split into train/test (80–20)
- Apply feature scaling
- Train:
 - Logistic Regression
 - KNN (k=5)
- Make predictions

Display

- Accuracy of both models
- Confusion Matrix
- Classification Report

Analysis Questions

- Which model performed better? Why?
- How does scaling affect KNN performance?
- Which class is most misclassified?

2. *Effect of Feature Scaling*

Analyze the importance of feature scaling.

Requirements

- Train Logistic Regression:
 - Without scaling
 - With scaling
- Use same train-test split

Display

- Accuracy (with vs without scaling)

Analysis Questions

- Why does scaling improve performance?
- Which algorithms require scaling most?

3. *Regression Model Comparison*

Predict house prices using regression models.

Requirements

- Use `fetch_california_housing()` dataset
- Split dataset
- Apply feature scaling
- Train:
 - Linear Regression
 - Decision Tree Regressor

Display

- MSE, RMSE, R^2 Score for both models

Analysis Questions

- Which model performed better?
- Why does Decision Tree behave differently from Linear Regression?

- Which metric is more interpretable (RMSE vs MSE)?

4. *Understanding Evaluation Metrics (Classification)*

Manually analyze classification performance.

Requirements

Using:

```
y_true = [0, 1, 0, 1, 0, 1, 1, 0]
y_pred = [0, 1, 0, 0, 0, 1, 1, 1]
```

- Compute:
 - Accuracy
 - Precision
 - Recall
 - F1 Score
- Generate confusion matrix
- Plot heatmap

Display

- All metric values
- Confusion matrix visualization

Analysis Questions

- When is accuracy misleading?
- When is precision more important than recall?
- Give a real-world example for high recall importance

5. *ROC Curve & Model Performance*

Evaluate classifier using ROC and AUC.

Requirements

- Use same dataset as Problem 4
- Compute:
 - ROC curve
 - AUC score
- Plot ROC curve

Display

- ROC graph
- AUC value

Analysis Questions

- What does AUC represent?
- What does AUC = 0.5 mean?
- Why is ROC better than accuracy for imbalanced data?

6. *Train-Test Split Importance*

Compare model performance with and without splitting.

Requirements

- Train model:
 - Using full dataset
 - Using train-test split
- Compare accuracy

Analysis Questions

- What is overfitting?

- Why should we never train on test data?
- What is data leakage?